ASSIGNMENT 11.2

2303A51930

BATCH-30

Task Description -1 – (Stack Using AI Guidance)
• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

CODE:

```
#create a Queue data structure following FIFO principles
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Peek from an empty queue")

    def size(self):
        return len(self.items)
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Front item:", queue.peek())  # Output: Front item: 1
    print("Queue size:", queue.size())  # Output: Queue size: 3
    print("Dequeued item:", queue.dequeue())  # Output: Dequeued item: 1
    print("Queue is empty:", queue.is_empty())  # Output: Queue is empty: False
```

```
    print("Dequeued item:", queue.dequeue())  # Output: Dequeued item: 2
    print("Dequeued item:", queue.dequeue())  # Output: Dequeued item: 3
    print("Queue is empty:", queue.is_empty())  # Output: Queue is empty: True
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026030601-win32-x64\bundled\libs\debugpy\launcher' '59060' '--' 'D:\AI Assisted coding\lab 11.2-1.py'
Front item: 1
Queue size: 3
Dequeued item: 1
Queue is empty: False
Dequeued item: 2
Dequeued item: 3
Queue is empty: True
```

Task Description -2 – (Queue Design)
• Task:Use AI assistance to create a Queue data structure following FIFO principles
Expected Output:
• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

CODE:

```
# design and implement a Stack data structure supporting basic stack operations.
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("Stack is empty")

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Stack is empty")

    def is_empty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
```

```
stack.push(2)
stack.push(3)
print("Top item:", stack.peek())  # Output: Top item: 3
print("Stack size:", stack.size())  # Output: Stack size: 3
print("Popped item:", stack.pop())  # Output: Popped item: 3
print("Stack is empty:", stack.is_empty())  # Output: Stack is empty: False
print("Popped item:", stack.pop())  # Output: Popped item: 2
print("Popped item:", stack.pop())  # Output: Popped item:
print("Stack is empty:", stack.is_empty())  # Output: Stack is empty: True
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026030601-win32-x64\bundled\libs\debugpy\launcher' '54892' '--' 'D:\AI Assisted coding\lab-11.2.py'
Top item: 3
Stack size: 3
Popped item: 3
Stack is empty: False
Popped item: 2
Popped item: 1
Stack is empty: True
```

Task Description -3 –(Singly Linked List Construction)

• Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE:

```
#build a singly linked list supporting insertion and traversal.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def traverse(self):
        current_node = self.head
        while current_node:
            print(current_node.data)
            current_node = current_node.next
```

```
# Example usage:
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    print("Linked List Traversal:")
    linked_list.traverse()  # Output: 1 2 3
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026030601-win32-x64\bundled\libs\debugpy\launcher' '54971' '--' 'D:\AI Assisted coding\lab-11.2.3.py'
Linked List Traversal:
1
2
3
```

Task Description -4 – (Binary Search Tree Operations)
• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.
Expected Output:
• BST program with correct node insertion and in-order traversal output.

CODE:

```
#implement the binary search tree and implement the following functions like insert, delete, search, inorder traversal, preorder traversal, postorder traversal
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
```

```
        else:
            self._insert_recursively(node.right, key)

    def search(self, key):
        return self._search_recursively(self.root, key)

    def _search_recursively(self, node, key):
        if node is None or node.val == key:
            return node
        if key < node.val:
            return self._search_recursively(node.left, key)
        return self._search_recursively(node.right, key)

    def inorder_traversal(self):
        return self._inorder_recursively(self.root)

    def _inorder_recursively(self, node):
        res = []
        if node:
            res = self._inorder_recursively(node.left)
            res.append(node.val)
            res = res + self._inorder_recursively(node.right)
        return res

    def preorder_traversal(self):
        return self._preorder_recursively(self.root)

    def _preorder_recursively(self, node):
        res = []
        if node:
            res.append(node.val)
            res = res + self._preorder_recursively(node.left)
            res = res + self._preorder_recursively(node.right)
        return res

    def postorder_traversal(self):
        return self._postorder_recursively(self.root)

    def _postorder_recursively(self, node):
        res = []
        if node:
            res = self._postorder_recursively(node.left)
            res = res + self._postorder_recursively(node.right)
            res.append(node.val)
        return res
# Example usage:
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
```

```
bst.insert(2)
bst.insert(4)
bst.insert(6)
bst.insert(8)

print("Inorder Traversal:", bst.inorder_traversal())  # Output: [2, 3, 4, 5, 6, 7, 8]
print("Preorder Traversal:", bst.preorder_traversal())  # Output: [5, 3, 2, 4, 7, 6, 8]
print("Postorder Traversal:", bst.postorder_traversal())  # Output: [2, 4, 3, 6, 8, 7, 5]
print("Search for 4:", bst.search(4) is not None)  # Output: True
print("Search for 10:", bst.search(10) is not None)  # Output: False
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026030601-win32-x64\bundled\libs\debugpy\launcher' '54995' '--' 'D:\AI Assisted coding\lab 11.2.4.py'
Inorder Traversal: [2, 3, 4, 5, 6, 7, 8]
Preorder Traversal: [5, 3, 2, 4, 7, 6, 8]
Postorder Traversal: [2, 4, 3, 6, 8, 7, 5]
Search for 4: True
Search for 10: False
```

Task Description -5 – (Hash Table Implementation)
• Task: Create a hash table using AI with collision handling
Expected Output:
• Hash table supporting insert, search, and delete using chaining or open

CODE:

```
#Create a hash table with collision handling using chaining
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair

    def search(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value associated with the key
        return None  # Key not found
```

```
    def delete(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True
        return False  # Key not found
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable(10)
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print("Search for 'name':", hash_table.search("name"))  # Output: Alice
    print("Search for 'age':", hash_table.search("age"))    # Output: 30
    print("Search for 'city':", hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print("Search for 'age' after deletion:", hash_table.search("age"))  # Output: None
    hash_table.insert("name", "Bob")  # Update existing key
    print("Search for 'name' after update:", hash_table.search("name"))  # Output: Bob
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugp
y-2025.19.2026030601-win32-x64\bundled\libs\debugpy\launcher' '64532' '--' 'D:\AI Assisted coding\lab 11.2.5.py'
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Search for 'age' after deletion: None
Search for 'name' after update: Bob
```